

CS 3510 Algorithms: Approximation Algorithms

Aaron Hillegass
Georgia Tech

When Optimal Requires Exponential Time

Give up on optimal.

We want to find a polynomial time algorithm that gives a good solution that is *guaranteed* to be within a certain factor of the optimal solution.

(Greedy algorithms are a pretty common approach.)

The guarantees are the hard part.

Load Balancing for soonest completion

- You have m machines: $\{M_1, M_2, \dots, M_m\}$.
- You have n jobs requiring different amounts of time: $\{t_1, t_2, \dots, t_n\}$.
- Assign jobs to machines: Let $A(i)$ be the set of all jobs assigned to machine M_i .
- The total time for all jobs assigned to machine M_i is $T_i = \sum_{j \in A(i)} t_j$.
- Minimize the maximum total time across all machines.
- (This problem is NP-hard.)

What would be a reasonable greedy algorithm?

Greedy Algorithm for Load Balancing

- Sort the jobs in descending order of their processing times
- Step through sorted list of jobs:
 - Figure out which machine currently has the smallest total processing time.
 - Assign the job to that machine.
- If optimal assignment A^* has a maximum total time of T^* .
 - $T^* \geq \frac{1}{m} \sum_{i \in 1 \dots n} t_i$
 - $T^* \geq \max_{i \in 1 \dots n} t_i$

What is the worst that our approximation can do?

Proof Greedy Algorithm is a 2-approximation 6/27

We need to show that our solution $T \leq 2T^*$.

Reminder: $T^* \geq \frac{1}{m} \sum_i t_i$ and $T^* \geq \max_i t_i$

When we added last job to each machine, it had the smallest total processing time. So, the every machine had less than $(\frac{1}{m}) \sum_i t_i$.

The last job added to each machine had time $t_i \leq \max_i t_i$

$$T_j \leq \left\lceil \frac{1}{m} \sum_i t_i \right\rceil + \lceil \max_i t_i \rceil \leq 2 \max\left(\left(\frac{1}{m}\right) \sum_i t_i, \max_i t_i\right) \leq 2T^*$$

Did we ever use the fact that we sorted
the jobs in descending order of their
processing times?

Can we get a tighter upper bound?

Sorting to show a $\frac{3}{2}$ -approximation

If $n \leq m$, our solution $T = T^*$ (each machine gets one task) else:

- $T^* \geq 2t_{m+1}$
- Let t_j be the last job assigned to a machine that defines T . By the assumption, $j \geq m + 1$, thus $t_j \leq t_{m+1} \leq \frac{1}{2}T^*$

$$T \leq \left\lceil \frac{1}{m} \sum_i t_i \right\rceil + t_j \leq \left\lceil \frac{1}{m} \sum_i t_i \right\rceil + \frac{1}{2}T^* \leq \frac{3}{2}T^*$$

Find Central Locations

- You have n sites: $S = \{s_1, \dots, s_n\}$
- You have distance function d :
 - $d(a, b) = d(b, a) \geq 0$
 - $d(a, a) = 0$
 - $d(a, b) \leq d(a, c) + d(c, b)$

Given an integer k , find $C \subset S$ such that

- $|C| = k$ and
- the required radius $r(C) = \max d(s, C)$ is minimized.

What's a reasonable greedy algorithm
for this problem?

Greedy Algorithm for Finding Central Locations^{11/27}S

- Pick any site s_0 , put it in C .
- While $|C| < k$:
 - Find the site s with $\max d(s, C)$
 - Add s to C

Proof: Greedy Algorithm is a 2-Approximation^{12/27}

To prove: If C^* is an optimal solution, $r(C) \leq 2r(C^*)$

Proof: Define r^* to be the radius of C^* . Obviously true if $k \geq |S|$.

Assume that we get a set C from the greedy algorithm, but $r(C) > 2r^*$. That is, there is a site s such that $d(s, C) > 2r^*$.

Imagine a single step of the algorithm: You are adding a site c' to C' .

$$d(c', C') \geq d(s, C') \geq d(s, C) > 2r^*$$

So every c' more than $2r^*$ away from every point in C . And s is more than $2r^*$ away from every point in C .

There is no way to cover $k + 1$ points more than $2r^*$ from each other with k circles of radius r^*

The proof works even if C^* is not restricted to the given sites.

Weighted Set Cover

There is a set U and a set of n subsets $S_1, \dots, S_n$.

Each subset S_i has a non-negative weight w_i .

Problem: Find a cover C of U such that the total cost:

$$\sum_{S_i \in C} w_i$$

is minimized.

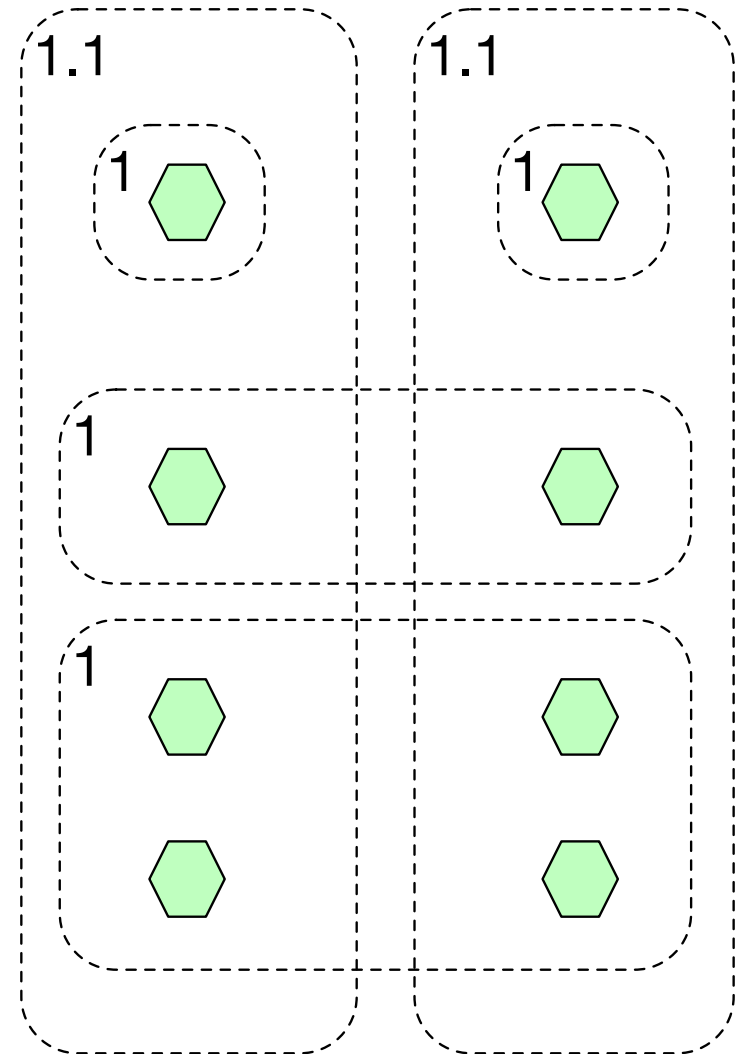
What is a reasonable greedy algorithm?

Greedy Algorithm for Weighted Set Cover

- Let $R = U$, Let C be the empty set.
- While $|R| > 0$:
 - Find the subset S_i with $\min \frac{w_i}{|S_i \cap R|}$
 - Add S_i to C
 - Remove elements of S_i from R

Note we can talk about the cost of picking up each element s :

For s in S_i , let $c_s = \frac{w_i}{|S_i \cap R|}$



$$H(n) = \sum_{i=1}^n \frac{1}{i}$$

Think of it as an approximation of the area under the curve $y = \frac{1}{x}$ from $x = 1$ to $x = n$.

$\ln(n) < H(n) < \ln(n) + 1$, so

$$H(n) \in \Theta(\ln(n))$$

Relating H to c_s

For any set S_k , $\sum_{s \in S_k} c_s \leq H(|S_k|)w_k$

Proof: Let $d = |S_k|$.

Make a list of all elements in S_k in order by which they are added to R .

s_j be an element added at some step j . At the start of that step $s_j, s_{j+1}, \dots, s_d \in R$

$$|S_k \cap R| \geq d - j + 1 \text{ so } \frac{w_k}{|S_k \cap R|} \leq \frac{w_k}{d-j+1}$$

Some may have been covered earlier in some S_i with $i < j$, so

$$c_{s_j} \leq \frac{w_i}{|S_i \cap R|} \leq \frac{w_j}{|S_j \cap R|} \leq \frac{w_k}{|S_k \cap R|} \leq \frac{w_k}{d-j+1}$$

Add up the costs of everything:

$$\sum_{s \in S_k} c_s \leq \sum_{j=1}^d c_{s_j} \leq \sum_{j=1}^d \frac{w_k}{d-j+1} = \frac{w_k}{d} + \frac{w_k}{d-1} + \dots + \frac{w_k}{1} = H(d)w_k$$

The Punch Line

Let $d^* = \max_i |S_i|$

Let C^* be the optimal solution and let w^* be the cost of it.

The cost of the greedy algorithm is at most $H(d^*)w^*$.

$$w^* = \sum_{S_i \in C^*} w_i \geq \sum_{S_i \in C^*} \frac{1}{H(d^*)} \sum_{s \in S_i} c_s \geq \frac{1}{H(d^*)} \sum_{s \in U} c_s = \frac{1}{H(d^*)} \sum_{S_k \in C} w_k$$

Pricing Model: Think about how each step contributes to the overall cost.

Weighted Vertex Cover

Given a graph $G = (V, E)$ and a weight w_i for each vertex i ,

Find a $C \subset V$ such that every edge touches a vertex in C .

Minimize the total weight: $\sum_{i \in C} w_i$

What is an algorithm that will get you a good result in polynomial time?

Thinking Prices

Every vertex i has a weight w_i . Every edge e will have a price p_e .

We say a price is fair if $w_i \geq \sum_{(i,j) \in E} p_{i,j}$

For any cover S and a set of nonnegative and fair prices:

$$\sum_{e \in E} p_e \leq \sum_{i \in S} w_i$$

Proof: $\sum_{i \in S} \sum_{e=(i,j)} p_e \leq \sum_{i \in S} w_i$

Some edges could be covered by multiple vertices, but each edge is counted at least once:

$$\sum_{e \in E} p_e \leq \sum_{i \in S} \sum_{e=(i,j)} p_e$$

$$\sum_{e \in E} p_e \leq \sum_{i \in S} w_i$$

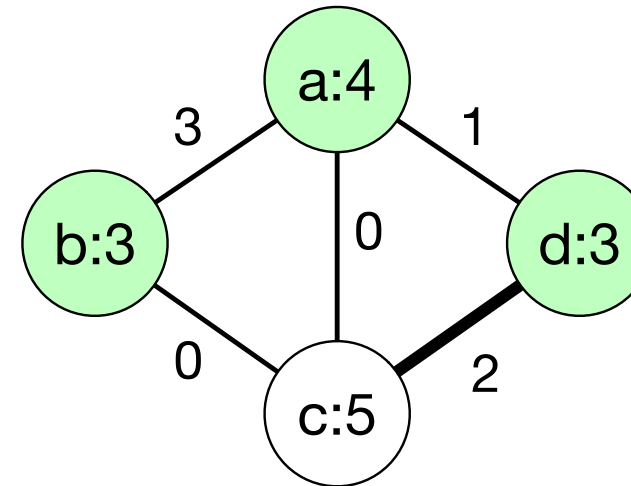
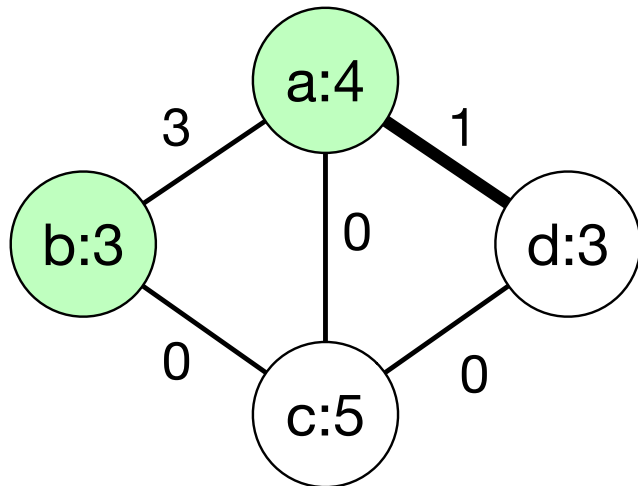
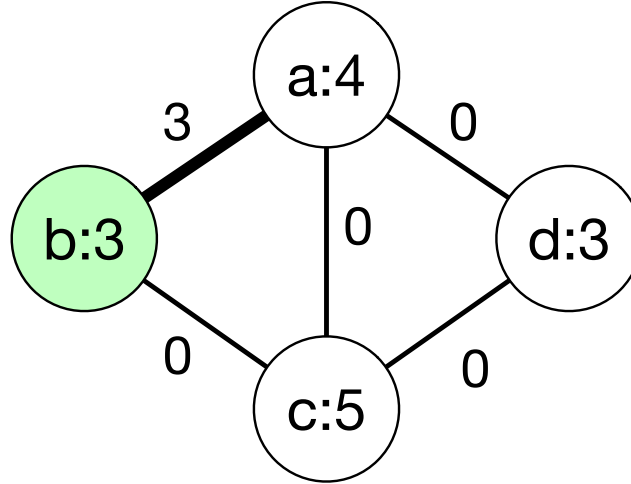
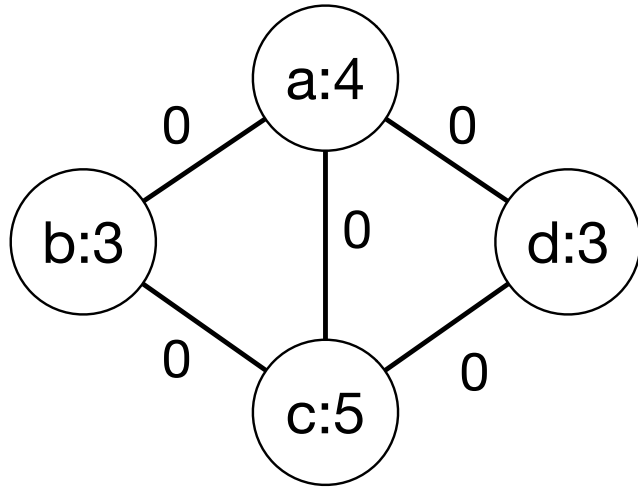
Algorithm

We say a vertex i is *paid for* if $\sum_{e=(i,j)} p_e = w_i$

- Set $p_e = 0$ for all edges.
- While there is an edge where neither endpoint is paid for:
 - Choose such an edge e .
 - Increase p_e until one of its endpoints is paid for
- Return all the vertices that are paid for

Example

23/27



Prices and Cover from Algorithm

The cover S and the prices p_e generated by the algorithm satisfy:

$$\sum_{i \in S} w_i \leq 2 \sum_{e \in E} p_e$$

Proof: Every vertex in S is paid for:

$$\sum_{i \in S} w_i = \sum_{i \in S} \sum_{e=(i,j)} p_e$$

Each edge can be counted at most twice:

$$\sum_{i \in S} w_i = \sum_{i \in S} \sum_{e=(i,j)} p_e \leq 2 \sum_{e \in E} p_e$$

S is a cover and its cost is at most twice the minimum cost of any cover.

It is a cover because every edge has at least one end that is paid for and in S .

Let S^* be an optimal cover, we know $\sum_{e \in E} p_e \leq w(S^*)$.

$$w(S) \leq 2 \sum_{e \in E} p_e \leq 2 \sum_{i \in S^*} w_i$$

That's The End

- This was a lot for one summer:
 - Divide and conquer
 - Graph algorithms
 - Dynamic programming
 - Complexity theory and NP-Completeness
 - MaxFlow and MinCut, Linear Programming, Cryptography
 - Approximation Algorithms
- Next: CS 4510: Automata and Complexity
- You did great. Thank you!
- Keep in touch.

Questions?

Slides by Aaron Hillegass

Based on lectures by Abraham Ladha